Development Team Design Document

Performance & Resource Insight System for Monitoring
~ PRISM~

MSCS710 Project
Spring 2026

This design document is prepared for use by the PRISM development team. Its purpose is to provide guidance for the design, build, and test of the product, ultimately enabling a successful deployment for the customer.

Version History

Version	Authors	Description	Date
1.0	Reece Schenck, Elizabeth Herrera, Roman Huerta	Initial Version	Feb 16, 2026

Contents

Introduction 4

Overview 4

Approach 4

Development Methodology 5

Computing Resources 5

Development 5

Metrics Gathering 5

Database Configuration 6

Database Schema 6

User Interface 8

PRISM System Dashboard Concept 9

Packaging & Installation 10

Accessibility 11

Supporting Material 11

Risks & Dependencies 11

GitHub 12

References 12

Introduction

Performance & Resource Insight System for Monitoring (PRISM) is a web based monitoring application designed to collect, store, and present system performance metrics. PRISM gathers operating system data related to CPU utilization, memory consumption, disk usage, and active processes. These metrics are stored in a relational database and displayed through a web interface that allows the user to monitor system data in real time and view trends. PRISM uses Node.js for backend services, React for the user interface, and MySQL for storage. The architecture follows a client-server model where the metric collection and storage occurs on the server and the user interacts with the application through a web browser. The objective of PRISM is to provide users with more insight into their system performance.

Overview

The following topics are covered in this document.

1. Approach
    - Describes the system architecture at a high level and explains how the major components interact.
2. Development Methodology
    - Outlines the iterative process used to build and refine the system.
3. Computing Resources
    - Defines the hardware, software, and tools required for development and deployment.
        - Nodejs, React, MySQL, Powershell, Python, Cron, VS Code

Approach

PRISM is designed as a client-server architecture. Its primary function is to collect system performance metrics, store them in a structured database, and present them through a web-based dashboard.

The system consists of three major components:

1. **Metric Collection**– Responsible for retrieving CPU, memory, disk, and process information from the host operating system.

2. **Backend API Layer** – A Node.js service that communicates with the MySQL database and exposes RESTful endpoints for metric retrieval.

3. **Frontend Presentation Layer** – A React-based Single Page Application that displays system metrics in a dashboard.

Development Methodology

PRISM will be developed using agile methodologies. With agile, this development will be done in stages, with each stage delivering different components that can be validated and tested. The initial phase of this project is focusing on establishing core functionality and structure, this includes database design and basic metric collection. Later phases will include adding backend APIs and front end dashboard components. Version control will be maintained through a GitHub repository to track changes and have backups. Function testing will be done to validate the metrics being collected are accurate and to ensure successful integration between the MySQL database and the React interface.

Computing Resources

This project requires both development and deployment environments.

Deployment Environment

The deployment server will include Node.js, MySQL, and the OS in which the metrics are collected. The server hosts the metric collection process, backend API services, and the database. Users access the system through a standard web browser.

Development Environment

The development environment includes

- Node.js
- MySQL database server
- React
- VS Code for software development
- GitHub for source code management

Development

Metrics Gathering

The purpose of this application is to provide a way for users to monitor the performance of their system. The metrics that have been identified as key to this solution are: CPU, Memory, and Disk utilization. Additionally, the user will be provided with the top twenty processes using the most memory on the system, to assist with investigations and system performance tuning.

Various performance indicators will be read using the following native Linux commands:

- lscpu
- cat /proc/meminfo

- df -h --total
- top

The use of native Linux functionality to obtain desired system information means PRISM is less likely to have future compatibility concerns than if it relied on a third party package.

The Python language will be used to read metrics from the LinuxOS and write them to a mySQL database. Data frames will be used to manipulate data into tables that can be pushed to the database using an ODBC connection.

Cron will be used to schedule and run the Python script at predetermined intervals. The cron daemon is a job scheduler in Linux, with jobs specified in crontab text files in /etc/crontab. The job will automatically be kicked off at system boot.

Cron has its own built-in logging, which will inform the customer that a collection operation was run. Additional logging will be created in the python script to show the outcome of attempts to read the system metrics and write them to the database.

Database Configuration

A MySQL database has been selected to store the system metrics. MySQL was chosen since it is free and open source software, and part of the ubiquitous LAMP (Linux, Apache, mySQL, PHP/Perl/Python) stack; meaning its components will be widely known and available, and the development environment mirrored by many customer environments. The database will contain four tables, one for each of the system commands being used. The tables will each use a timestamp as their unique identifier. At this time, we do not envision the need for views in the 6 database, since data will be aggregated in real time for the web app. The data in these tables will be periodically pruned, so as not to exceed a history further back than web client views permit, or customer needs dictate.

Database Schema

The PRISM server database uses the following mySQL tables. The primary key for each table is the same [dateTime], and is shown once for all tables.

Table primary keys are same for all tables:

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	date Time	0	A	No	

CPU Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra

1	**dateTime**	datetime			No	None		
2	**cpuMHz**	decimal(8,3)			No	None		
3	**cpuMHzMin**	decimal(8,3)			No	None		
4	**cpuMHzMax**	decimal(8,3)			No	None		

Memory Info Table:

#	**Name**	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**dateTime**	datetime			No	None		
2	**MemTotal**	int(10)			No	None		
3	**MemFree**	int(10)			No	None		

Disk Info Table:

#	Name	Type	Collation	Attributes	Nul l	Default	Comments	Extra
1	**dateTime**	datetime			No	None		
2	**size**	decimal(5,1)			No	None		
3	**used**	decimal(5,1)			No	None		
4	**avail**	decimal(5,1)			No	None		
5	**usePercent**	tinyint(3)			No	None		

Process Info Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**dateTime**	datetime			No	None		
2	**pid**	mediumint(8)			No	None		
3	**user**	varchar(20)	utf8_unicode_ci		No	None		
4	**cpuPercent**	decimal(4,0)			No	None		
5	**memPercent**	decimal(4,0)			No	None		

6	**command**	varchar(40)	utf8_unicode_ci		No	None		

Users Tables:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**user**	varchar(20)	utf8_unicode_ci		No	None		
2	**password**	varchar(256)	utf8_unicode_ci		No	None		
3	**admin**	char(1)	utf8_unicode_ci		No	None		
4	**email**	varchar(64)	utf8_unicode_ci		No	None		
5	**phone**	varchar(20)	utf8_unicode_ci		No	None		
6	**description**	varchar(50)	utf8_unicode_ci		No	None		

The database will additionally contain a table of user logins. This table will be used to authenticate access to the web application, and to program administrative tasks. This table will be the only database component that accepts user created, vs system generated data. To limit direct database access, and prevent insertion of malicious instructions into code being executed [SQL injection], all user input will be sanitized prior to use, and parameterized prepared queries will be used for all user-initiated database access, as this binds data to a prepared SQL statement at query runtime.

User Interface

The user interface is designed as a web-based application rather than a native desktop or mobile app. This approach offers significant advantages:

- Accessibility: Administrators can monitor the system from any device (laptop, tablet, or smartphone) with a standard web browser.
- Zero Client Installation: Users do not need to install specific software on their local machines to view the data; they simply navigate to the server's URL.

Backend API (Node.js)

The Node.js server acts as the middleware, querying the MySQL database and exposing data via RESTful endpoints.

- GET /api/metrics/latest: Returns the single most recent snapshot for real-time gauges.

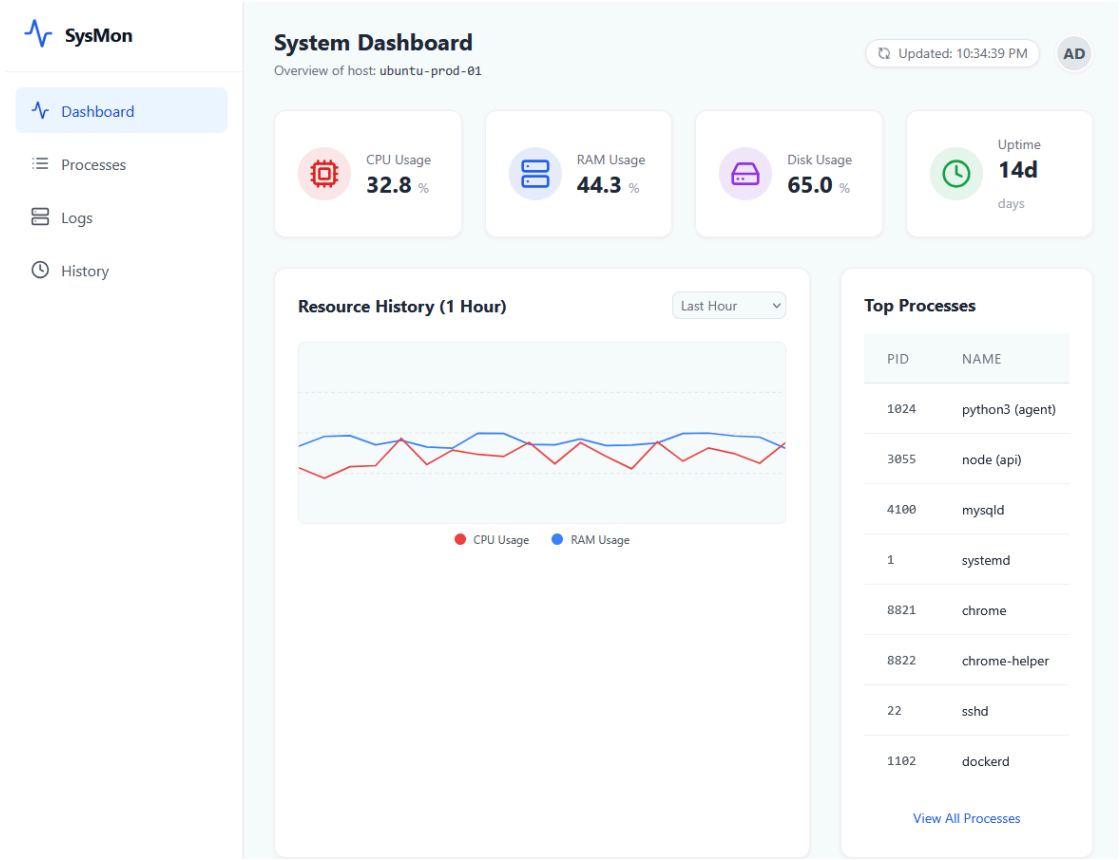
- GET /api/metrics/history?range=1h: Returns a list of snapshots from the last hour for historical line charts.
- GET /api/processes/top: Returns the top 10 processes by CPU usage for the process table.

Frontend Architecture (React)

The User Interface is a Single Page Application (SPA) built with React. It consists of the following key components:

- DashboardContainer: The main layout wrapper.
- MetricGauge: A component to display current percentages (CPU, RAM, and Disk usage).
- HistoryChart: Uses a charting library (e.g., Recharts) to visualize the JSON data from the history endpoint.
- ProcessTable: A sortable data table showing active processes and their resource consumption.

PRISM System Dashboard Concept



Sample dashboard generated using Gemini

Packaging & Installation

Packaging Format & Creation

Since the target OS is Ubuntu, the application will be distributed as a Debian Package (.deb). This format is the standard for Debian-based systems and ensures consistent deployment.

- **Build Tool:** The package will be created using the dpkg-deb utility.
- Creation Process:
    1. Prepare a build directory structure (e.g., build/usr/local/bin, build/etc/systemd/system).
    2. Place executable files and configuration in the respective folders.
    3. Create the DEBIAN/control file defining metadata (version, dependencies).
    4. Execute the build command: dpkg-deb --build linux-monitor_1.0_amd64

Installation via Package Manager

The client installation is handled by the system's package manager, ensuring that all dependencies are resolved automatically.

- **Primary Tool:** apt (Advanced Package Tool).
- **Command:** sudo apt install ./linux-monitor_1.0_amd64.deb
- **Process:** apt will read the control file, download required libraries (like Python or Node.js if missing), copy files to the correct locations, and register the systemd service.

Package Dependencies

The package metadata will define the following dependencies to prevent "Dependency Hell":

- python3 (>= 3.8)
- python3-mysql.connector (Required for the Python agent to communicate with the database)
- mysql-server
- nodejs (>= 14.x)

Installation Scripts (Maintainer Scripts)

The package includes hooks to automate setup and cleanup:

- postinst script (Post-Installation):
    1. Create the MySQL database schema and user.
    2. Install Node.js dependencies (npm install in the app directory).
    3. Reload systemd daemon, enable, and start the linux-monitor service.
- prerm script (Pre-Removal):
    1. Stop the service using systemctl.
    2. Disable the service unit.

Accessibility

To ensure the dashboard is usable by administrators with disabilities, the frontend will comply to WCAG 2.1 Level AA guidelines:

- High Contrast Visualization: Charts use high-contrast color palettes and patterns to assist users with color vision deficiencies (e.g., distinguishing CPU vs RAM lines).
- Screen Reader Support: Semantic HTML5 and ARIA labels are used throughout the application. Complex SVG charts include accessible descriptions or fallback tabular data for screen readers.
- Keyboard Navigation: The entire dashboard, including data tables and filter dropdowns, is fully navigable using standard keyboard controls (Tab, Enter, Arrow Keys).

Supporting Material

Comprehensive documentation will be provided to end-users and administrators to ensure effective system operation. This documentation will include:

- Installation Guide: Detailed instructions on how to install the application using the Debian Package (.deb). This includes command-line examples and troubleshooting steps for common installation issues.
- Execution Manual: A guide on how to start, stop, and restart the background service (Daemon) using systemd controls, as well as instructions for executing the portable version manually.
- Feature Reference: A full explanation of the program's features, including how to read the Dashboard metrics (CPU, RAM, Disk), how to interpret the process list data, and how to utilize historical data charts for system analysis.

Risks & Dependencies

Critical Dependencies

- Linux Kernel Dependence: The entire data acquisition layer is tightly coupled to the Linux /proc pseudo-filesystem. This application is not portable to other operating systems (Windows, macOS, BSD) without a complete rewrite of the Agent component in case we would consider implementing the tool for other systems.
- Database Availability: The Python Agent requires a persistent connection to the MySQL server. If the database service stops, metrics collected during the downtime may be lost unless a local buffering mechanism is implemented.
- Root Privileges: Installation via the .deb package requires sudo access to configure systemd services and copy binaries to protected directories (/usr/local/bin, /etc).

Project Risks & Mitigations

The "Observer Effect":

- Risk: The monitoring agent itself might consume significant CPU or RAM, skewing the metrics it is attempting to measure.
- Mitigation: The Agent is designed to be lightweight, using efficient file I/O operations and avoiding heavy libraries. We will benchmark the agent to ensure it consumes < 1% CPU under normal load.

Data Storage Saturation:

- Risk: High-frequency polling (e.g., every second) generates massive amounts of data. Over time, the process_snapshots table could grow to millions of rows, filling the disk.
- Mitigation: A "Data Retention Policy" will be implemented. A scheduled database job (cron) will purge raw metrics older than 30 days or aggregate them into hourly averages to save space.

Security Vulnerabilities:

- Risk: Exposing system metrics via a web interface could leak sensitive information (e.g., running process arguments containing passwords) if access control is not enforced.
- Mitigation: The web dashboard will be restricted to the local network or require basic authentication (via Nginx reverse proxy) in a production environment.

GitHub

https://github.com/ElizabethHerrera12/MSCS_710L_711.git

References

https://www.w3.org/TR/WCAG21/